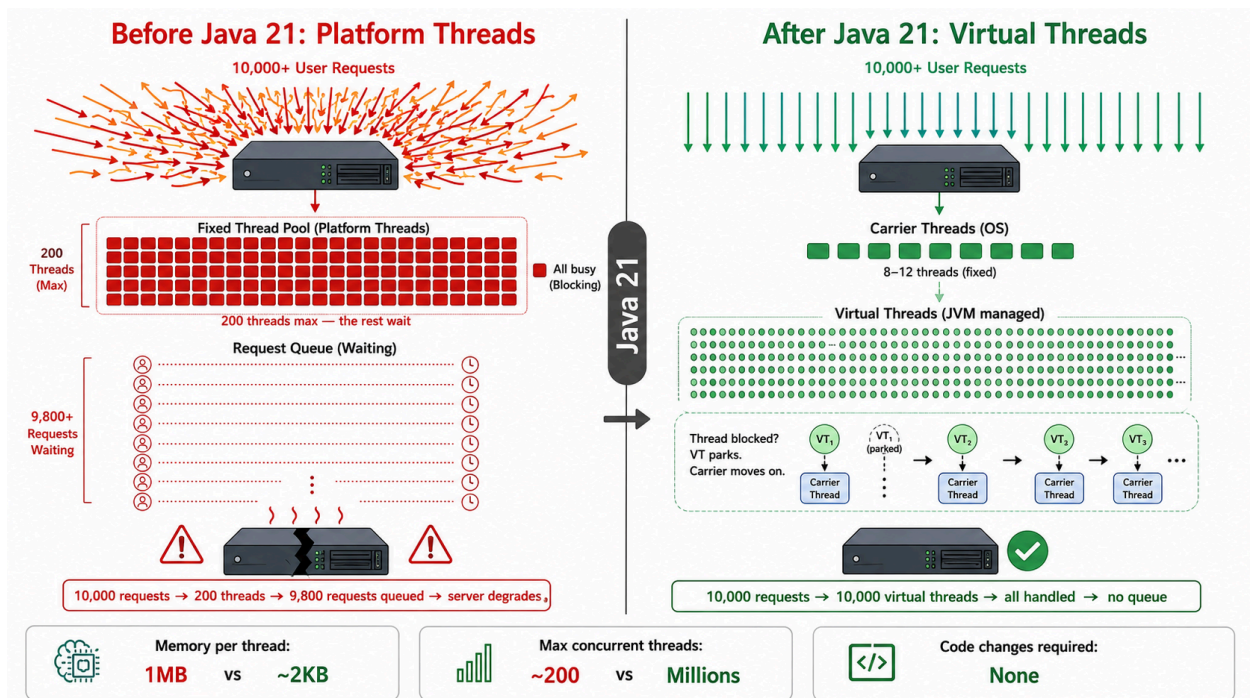


Virtual Threads

The Original Problem

Traditional Java web servers use a thread-per-request model. One request arrives, one OS thread handles it from start to finish.



```
// what happens inside your servlet container for every request
Thread thread = new Thread(() -> {
    User user = userRepository.findById(id); // thread blocks here, waiting for DB
    Order order = orderRepository.findByUser(user); // blocks again
    return buildResponse(user, order);
});
```

OS threads are expensive:

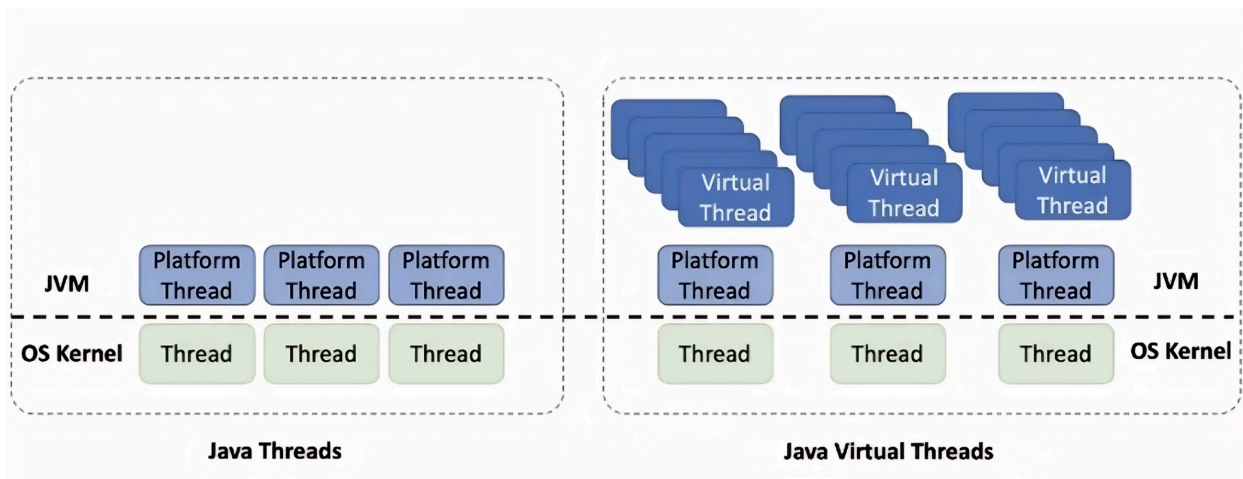
- Each one allocates roughly 1MB of stack memory by default
- Context switching between OS threads has significant CPU overhead
- A typical server can handle 200 to 500 concurrent OS threads comfortably before degrading

What are Virtual Threads

Virtual Threads are lightweight threads managed by the **JVM** instead of the operating system. Introduced as a stable feature in **Java 21**, they are part of **Project Loom**.

- **Platform Threads** (traditional): 1 thread = 1 OS thread (~1MB stack memory). Limited to a few thousand.
- **Virtual Threads**: Very cheap (~few KB), can create **millions** without exhausting resources.

They allow you to write simple **blocking** code that scales almost as well as reactive/non-blocking code.



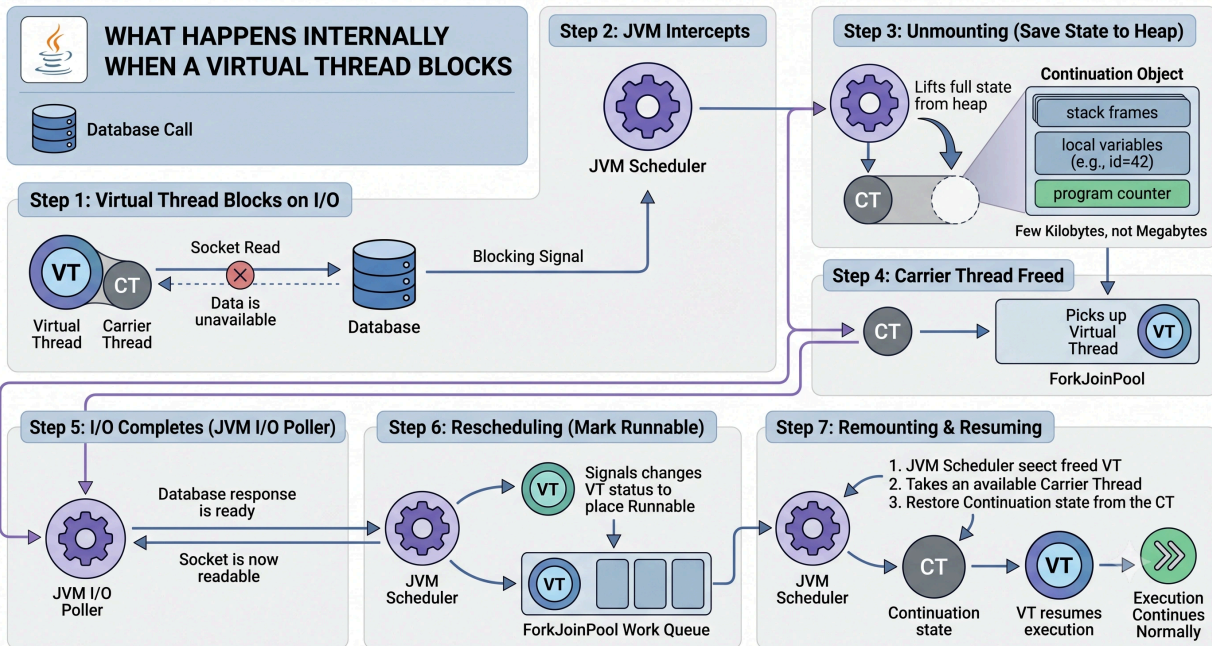
How do the Virtual Threads work

```
public UserResponse getUser(Long id) {
    User user = userRepository.findById(id); // JDBC blocks here
```

```

return UserResponse.from(user);
}

```



1. The Virtual Thread initiates a network I/O request. Because the data isn't ready, the execution halts.
2. Instead of blocking the underlying OS thread, the VT-aware networking layer catches the block and alerts the JVM scheduler.
3. The JVM freezes the thread, saving its stack frames, local variables, and program counter into a lightweight heap object costing only a few kilobytes.
4. The underlying OS/Carrier thread is instantly liberated, allowing the ForkJoinPool to assign it to other waiting workloads.
5. Once the database responds, the OS flags the socket as readable, which is instantly caught by the JVM's background I/O polling mechanism.
6. The JVM changes the virtual thread's status back to "Runnable" and pushes it straight back into the scheduler's execution queue.
7. A free carrier thread claims the VT, unpacks its saved stack from the heap, and resumes execution seamlessly right where it left off.

Metric	Traditional Threads (1:1 with OS)	Virtual Threads (M:N with OS)
Memory Cost	~1 MB per thread (allocated to OS stack).	A few KB per thread (allocated as an object on the Java Heap).
Blocking Cost	Context Switch: The OS has to pause the CPU, save registers, move the thread to a wait queue, and context-switch a new one in. Highly expensive.	Pointer Swap: The JVM simply unmounts the virtual thread, leaves its data on the heap, and points the Carrier Thread to the next item in the <code>ForkJoinPool</code> queue.
Limits	Hard capped by OS memory and scheduling overhead (usually a few thousand max).	Practically limited only by your available JVM Heap space (millions easily).

Enabling Virtual Threads in Spring Boot

Spring Boot 3.2 and above. One line of config.

```
spring:
  threads:
    virtual:
      enabled: true
```

Once this configuration is active, the **Spring Framework** automatically hooks into the runtime infrastructure to optimize thread usage across multiple core components:

- **Tomcat / Jetty / Undertow:** Spawns a new, lightweight virtual thread for each incoming HTTP request instead of taking a heavy platform thread from a pool.
- **Task Execution:** Upgrades the application's underlying `AsyncTaskExecutor` so that `@Async` methods and task-bound abstractions execute on virtual threads automatically.
- **Message Listeners:** Migrate consumer loops in blocking message workflows (such as RabbitMQ, Apache Kafka, and Redis) to a virtual-thread-per-task execution model.

```
// this code looks blocking, but on a virtual thread it is fine
@GetMapping("/users/{id}")
public UserResponse getById(@PathVariable Long id) {
    return userService.findById(id); // JDBC blocks here -- V
    // T unmounts, OS thread freed
}
```

Key Benefits of Virtual Threads

- **Simple imperative code** (no callbacks, no reactive operators)
- Excellent for **I/O-bound** work (HTTP calls, database, file I/O)
- Easy debugging and stack traces (normal threads)

Virtual Threads vs OS Threads

```
// OS thread -- expensive, limited
Thread osThread = new Thread(() -> {
    System.out.println("OS thread: " + Thread.currentThread());
});
osThread.start();

// Virtual thread -- cheap, millions possible
Thread virtualThread = Thread.ofVirtual().start(() -> {
    System.out.println("Virtual thread: " + Thread.currentThread());
});

// or via executor
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
executor.submit(() -> {
    // each task gets its own virtual thread
});
```

```
return userRepository.findById(1L);  
});
```

Installing Apache WorkBench

macOS

1. **ab** comes **pre-installed** on almost all Macs (because macOS includes Apache HTTP Server).
- Just open Terminal and type

```
ab -V
```

If it shows a version → it's already there.

2. If not found, install it easily with Homebrew:

```
brew install httpd
```

Windows

- **ab** is **NOT** pre-installed.
- You have to install it manually.

Easiest ways (recommended order):

1. **Best for teaching:** Use **WSL2** (Windows Subsystem for Linux)

- Install Ubuntu from Microsoft Store
- Then run:

```
sudo apt update  
sudo apt install apache2-utils -y
```

2. **Apache Lounge** (cleanest native way)

- Download from <https://www.apachelounge.com>
- Extract and use `ab.exe` from the bin folder.

```
ab -n 1000 -c 200 -s 300 http://localhost:8080/api/v1/benchmark/user/1
```

Apache Benchmark command